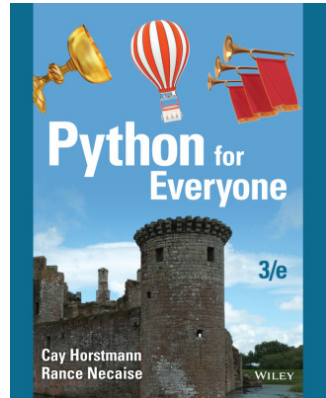


Lesson 1. Introduction: Computers, Python, and Algorithms

Based on Chapter 1 – *Python for Everyone*,
3rd Edition

Lecture information

- Course: Introduction to Python Programming
- Topics: computer programs, computer components, Python, common errors, algorithms
- Main reference: Cay Horstmann and Rance Necaise



Book cover used as a source reference

Learning objectives

After this lesson, students will be able to:

- explain computer programs and programming;
- describe the basic components of a computer;
- run their first Python program;
- analyze the Hello, World! program;
- distinguish compile-time errors, exceptions, and logic errors;
- describe algorithms using pseudocode.

Chapter 1 structure

- 1 Computer Programs
- 2 The Anatomy of a Computer
- 3 The Python Programming Language
- 4 Programming Environment
- 5 First Python Program
- 6 Errors
- 7 Algorithm Design and Pseudocode

Computers and programs

Computer program

A computer program is a sequence of **instructions** and **decisions** that asks the computer to perform a specific task.



Example task: ask the user for a name, print a greeting, and optionally print a welcome message.

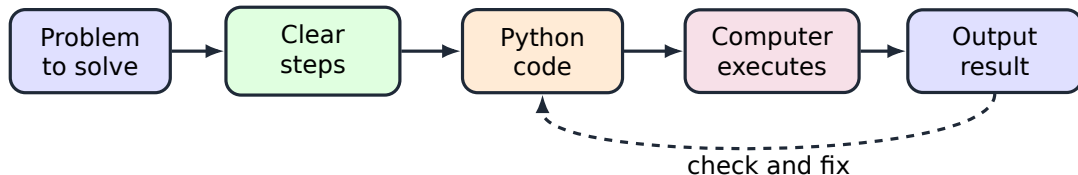
Key idea

The same computer can perform many different tasks because it can run many different programs.

What is programming?

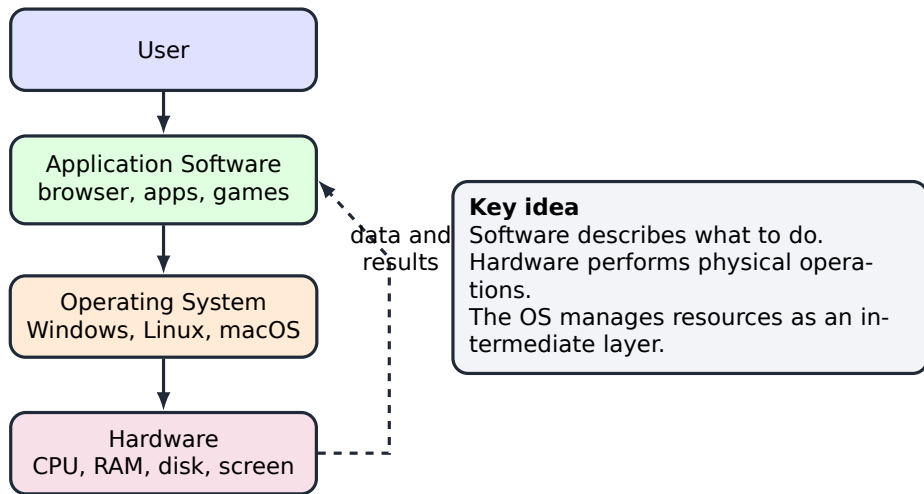
Programming

Programming is the act of **designing** and **implementing** computer programs.



- A computer does not guess intentions; it follows instructions.
- The programmer translates a problem into executable steps.

Hardware and Software



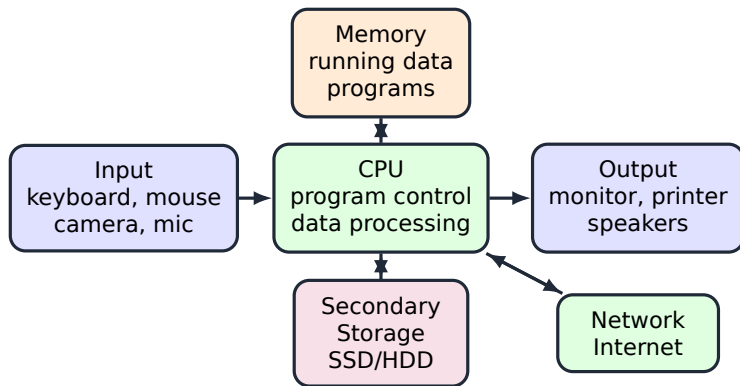
Self Check 1

- 1 A computer program is a sequence of _____ and decisions.
- 2 Programming is the process of designing and implementing computer programs.
True or false?
- 3 Why can the same computer perform many different kinds of tasks?

Answers to Self Check 1

- 1 Instructions.
- 2 True.
- 3 Because the hardware can run many different programs; each program describes a specific task.

Main components of a computer



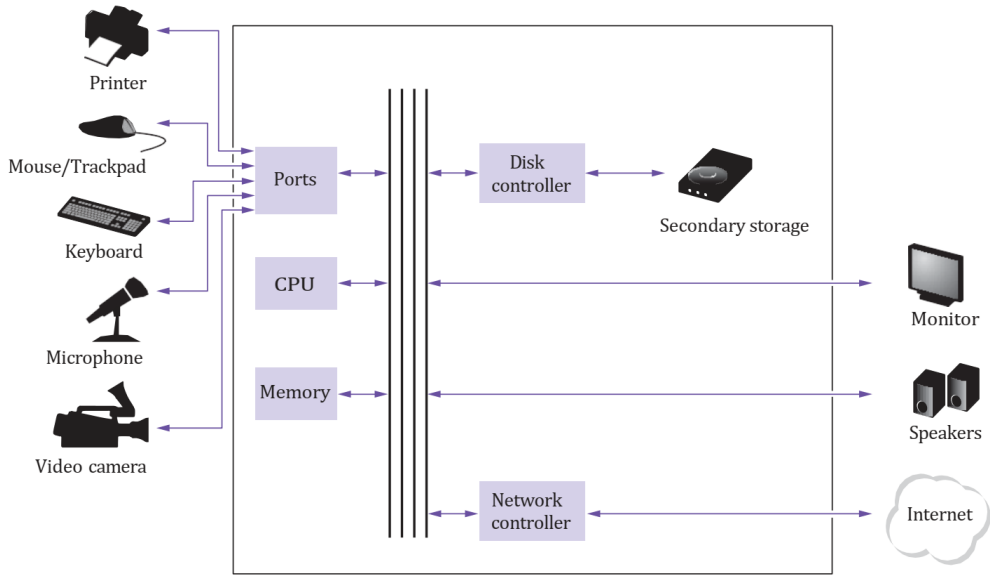


Figure 3 Schematic Design of a Personal Computer

CPU, memory, and storage

CPU Controls programs and processes data.

Memory Stores programs and data currently in use; fast but usually not persistent after power-off.

Secondary storage Provides long-term storage, such as hard drives, SSDs, and USB drives.

Key idea

When a program runs, its file is usually speakersded from secondary storage into memory so the CPU can read and execute it.

Input, output, and network

Input

Data enters the computer.

- keyboard
- mouse
- microphone
- camera

Output

Information produced by the computer.

- monitor
- speakers
- printer

Network

Exchanges data with other computers.

- local network
- Internet
- remote server

Computers Are Everywhere

Computers are not only laptops or PCs.

- Smartphones, bank cards, and transit cards.
- Modern cars, medical devices, factories, and robots.
- Data centers, social networks, and e-commerce.

Message

Understanding computers and programming helps students work better in many fields, not only information technology.

Self Check 2

- 1 Which component controls programs and processes data?
- 2 Secondary storage usually keeps data after power-off. True or false?
- 3 Keyboard and mouse belong to which device group?

Answers to Self Check 2

- 1 CPU.
- 2 True.
- 3 Input devices.

Python is a high-level programming language

High-level language

A language close to how humans describe problems; easier to read and write.

Examples: Python, Java, C++, JavaScript.

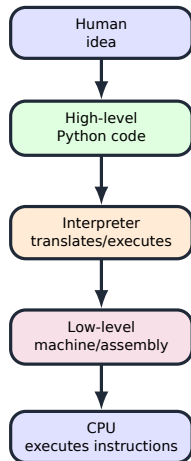
High-level command example: `print("Hello, World!")`

Low-level language

A language closer to hardware and the CPU; harder to read but gives more detailed machine control.

Examples: machine code, assembly language.

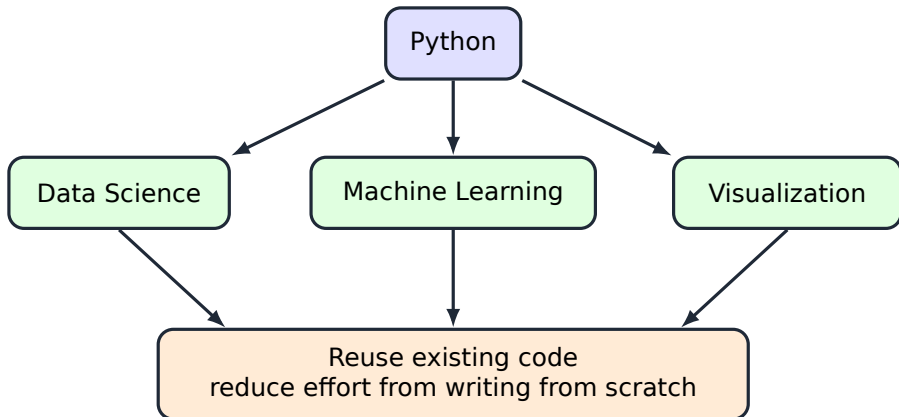
Low-level command example: `MOV AX, 1`



Packages in Python

Package

A package provides ready-made code for a specific problem domain.



Self Check 3

- 1 Is Python a high-level or low-level programming language?
- 2 Python can run on only one operating system. True or false?
- 3 How do packages help programmers?

Answers to Self Check 3

- 1 High-level.
- 2 False.
- 3 Packages provide ready-made code for different problem domains and help reuse existing solutions.

Getting familiar with the programming environment

A programming environment usually includes:

- Python interpreter;
- a text editor or IDE;
- terminal/command prompt;
- a folder for storing program files;
- tools for running and testing programs.

Note

Students should spend time getting familiar with the tools before working on larger assignments.

The first program

Create the file `hello.py`:

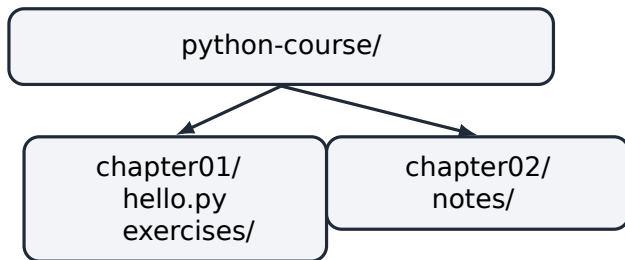
```
# My first Python program.  
print("Hello, World!")
```

Expected output:

```
Hello, World!
```

- Python is case-sensitive.
- `print` is different from `Print`.

Organizing the working folder



- Python files usually have the .py extension.
- It is useful to have a separate folder for each chapter/assignment.
- A backup strategy is needed.

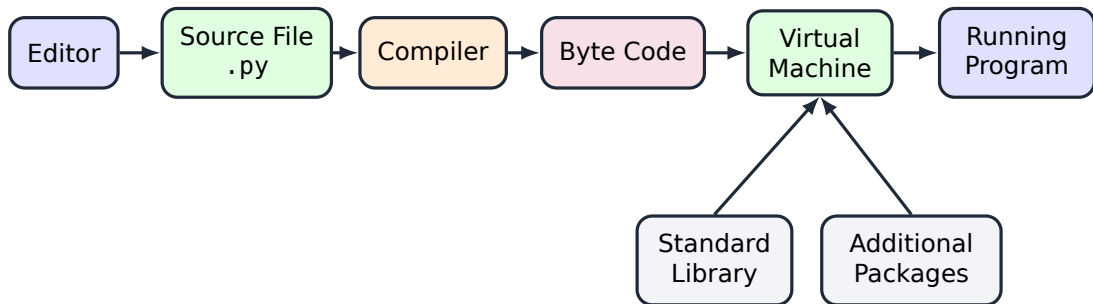
Interactive Mode

Interactive mode lets you run commands one by one and immediately see the results.

```
>>> print("Hello, World!")  
Hello, World!  
>>> 7035 * 0.15  
1055.25
```

- Quickly test expressions.
- Check new syntax.
- Use Python as a pocket calculator.

From source code to running program



Self Check 4

- 1 What extension do Python program files usually have?
- 2 Is Python case-sensitive?
- 3 What is the tool that reads and executes Python programs called?

Answers to Self Check 4

- 1 .py
- 2 Yes.
- 3 Python interpreter.

Analyzing the first program

```
# My first Python program.  
print("Hello, World!")
```

Comment starts with # and is used to explain code to readers.

Statement a command that asks Python to perform an action.

Function print is a built-in function.

Argument "Hello, World!" is the value passed to the function.

String a sequence of characters enclosed in quotation marks.

Strings and expressions

Compare these two statements:

```
print("39 + 3")  
print(39 + 3)
```

Output:

```
39 + 3  
42
```

- The first line prints the literal string.
- The second line evaluates the expression and prints the result.

The print syntax

```
print(value1, value2, ..., valuen)
```

Example:

```
print("The answer is", 6 + 7, "!")
```

- Values are printed in order.
- By default, they are separated by spaces.
- `print()` with no arguments prints a blank line.

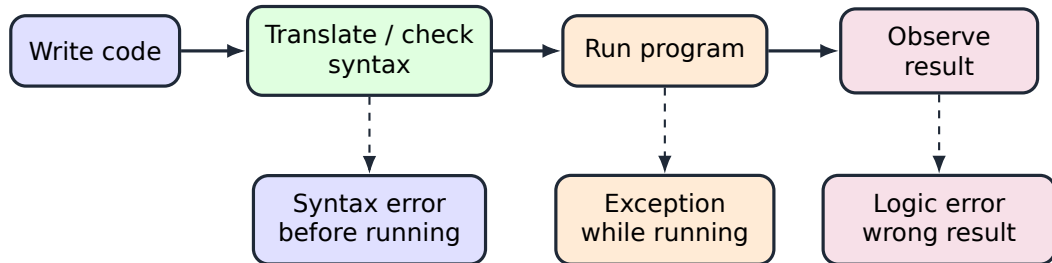
Self Check 5

- 1 What character begins a Python comment?
- 2 What is a value passed to a function called?
- 3 How are `print("39 + 3")` and `print(39 + 3)` different?

Answers to Self Check 5

- 1 The # character.
- 2 Argument.
- 3 The first statement prints the string `39 + 3`; the second evaluates the addition and prints 42.

Three common groups of errors



- The location of an error helps us know how to inspect and fix the program.

Examples of errors

Syntax error

```
print("Hello)
```

Missing quotation mark.

Exception

```
print(1 / 0)
```

Division by zero.

Logic error

```
print("Hello, Word!")
```

Runs, but does not match the intended result.

Common Error: Misspelling Words

```
Print("Hello, World!")
```

- Python case sensitive.
- Print is not the same as print.
- When an error is hard to understand, check spelling and capitalization.

Self Check 6

- 1 What is an error that violates syntax rules called?
- 2 What is an error that occurs when a syntactically correct statement cannot be executed?
- 3 Are Print and print the same in Python?

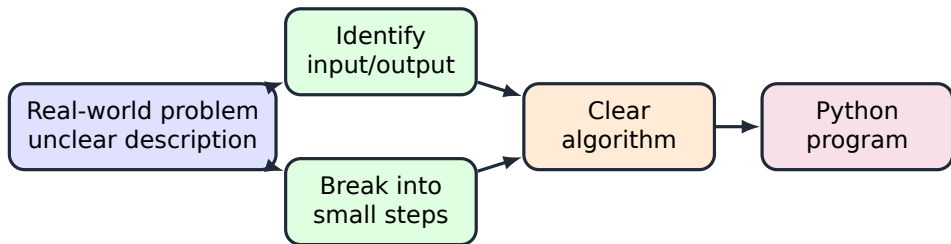
Answers to Self Check 6

- 1 A compile-time error or syntax error.
- 2 Exception.
- 3 No. Python is case-sensitive.

Problem-solving thinking

Before writing code

Understand the problem and describe the solution with clear steps.



Algorithm and pseudocode

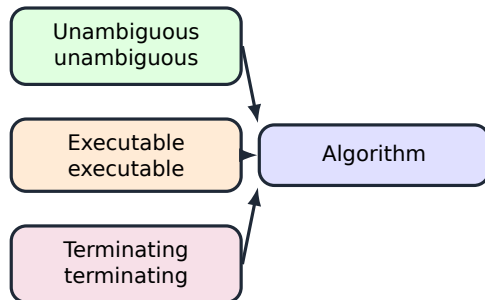
Algorithm

A sequence of steps for solving a problem; it should be:

- unambiguous;
- executable;
- terminating.

Pseudocode

A human-readable description of an algorithm before translating it into code.

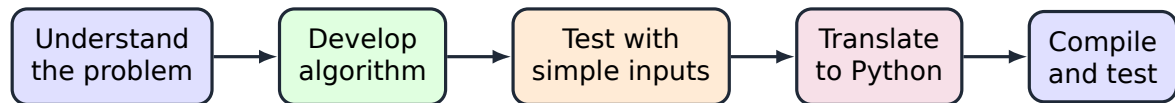


Pseudocode example: bank account

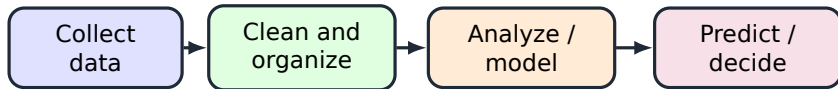
Problem: deposit 10,000 USD with 5% annual interest. How many years until the balance at least doubles?

```
Set year to 0
Set balance to 10000
While balance is less than 20000
    Add 1 to year
    Set interest to balance x 0.05
    Add interest to balance
Report year as the answer
```

A simple software development process



Data Is Everywhere



Examples: customer analytics, fraud detection, image recognition, medical diagnosis support, self-driving cars.

How To: write pseudocode

Problem: choose which car is more economical over 10 years.

For each car, compute total cost:

 annual fuel consumed = annual miles driven / fuel efficiency

 annual fuel cost = price per gallon x annual fuel consumed

 operating cost = 10 x annual fuel cost

 total cost = purchase price + operating cost

If total cost of car 1 < total cost of car 2

 Choose car 1

Else

 Choose car 2

Worked Example: tiling a floor

Problem: tile a floor with alternating black and white tiles, each 4×4 inches.

```
Place a black tile in the northwest corner
While the floor is not yet filled
  Repeat width / 4 - 1 times
    If previous tile was white
      Pick a black tile
    Else
      Pick a white tile
    Place the picked tile east of previous tile
  If there is space to the south
    Place opposite color below first tile of row
```

Self Check 7

- 1 What is pseudocode?
- 2 What three properties should an algorithm have?
- 3 Why should we test an algorithm with small examples before writing code?

Answers to Self Check 7

- 1 An informal description of the steps for solving a problem.
- 2 Unambiguous, executable, and terminating.
- 3 To detect errors in the idea before translating it into Python.

In-class practice exercises

- 1 Create `hello.py` and run the Hello, World! program.
- 2 Create a syntax error, read the error message, and fix it.
- 3 Compare `print("39 + 3")` and `print(39 + 3)`.
- 4 Open interactive mode and run a few arithmetic expressions.
- 5 Write pseudocode for splitting a restaurant bill.

Quick quiz

- 1 What extension do Python files usually have?
- 2 Is `print` a function or a string?
- 3 What symbols enclose a string?
- 4 What error usually occurs when dividing by zero while running?
- 5 Does pseudocode have to follow Python syntax exactly?

Answers to Quick quiz

- 1 .py
- 2 Function.
- 3 Single or double quotation marks.
- 4 Exception.
- 5 No.

Summary

- Program = instructions + decisions.
- CPU, memory, storage, input/output, and network components work together to run programs.
- Python is easy to learn, portable, and has many packages.
- The first program helps explain comments, functions, arguments, and strings.
- Errors are normal; we need to know how to read and classify them.
- Before coding, understand the problem and write the algorithm in pseudocode.